



Facultad de Ingeniería
Universidad de Concepción



Departamento de Ingeniería Informática
y Ciencias de la Computación
Universidad de Concepción

Universidad de Concepción
Departamento de Ingeniería Informática y Ciencias de la Computación
Introducción a la Computación Paralela
2026-1

Paralelización del Algoritmo de Tableau para la Lógica Descriptiva *ALC*

Informe Final

Estudiante: Antonio Skorin García
Docente: Cecilia Hernández
Ayudante: Álvaro Guzmán
Repositorio: <https://github.com/Askorin/alc-tableau-parallel>
Fecha: 15 de julio de 2026

Resumen

La clasificación de ontologías mediante el algoritmo de *tableau* es una tarea computacionalmente costosa cuyo método dominante sigue siendo, en la práctica, secuencial. En este trabajo se implementa un razonador de *tableau* paralelo para la lógica descriptiva $\mathcal{ALC}$ en C++ con *OpenMP*, capaz de procesar concurrentemente las ramas conjuntivas del árbol de expansión siguiendo la propuesta de Wu y Haarslev (2013a), y además incorporando una arquitectura de memoria basada en arenas *thread-local* con copia por prefijos que permite ajustar la granularidad del paralelismo. La solución se evaluó sobre el *benchmark* artificial *tea* del trabajo original y sobre fragmentos $\mathcal{ALC}$ de la ontología médica *Galen*, midiendo *speedup* y eficiencia para 1 a 4 hebras. Se obtuvo un *speedup* máximo de 2,54 con 4 hebras (2,53 con 3), superando los resultados del trabajo de referencia a igual número medio de ramas conjuntivas (τ), y se confirmó empíricamente que τ acota la escalabilidad alcanzable del AND-*parallelism*.

Índice

1. Introducción	1
1.1. Definición del Problema	1
1.1.1. Lógicas Descriptivas y Razonamiento	1
1.1.2. El Algoritmo de <i>Tableau</i>	2
1.2. Motivación e Importancia	3
2. Trabajo Previo y Soluciones Existentes	4
2.1. Paralelización AND/OR	4
2.2. Paralelización de Granularidad más Gruesa	4
2.3. Otros Paradigmas Concurrentes	5
2.4. Desafíos Identificados	5
3. Propuesta de Solución	6
3.1. El Algoritmo	6
3.2. Implementación en <i>OpenMP</i>	7
3.3. Manejo de Memoria	8
4. Metodología	9
4.1. Descripción de los Datos	9
4.1.1. <i>Tea Ontology</i> Artificial	9
4.1.2. Ontología <i>Galen</i>	10
4.1.3. Datos Finales	10
4.2. Baseline	11
4.3. Hiperparámetros	11
4.4. Métricas de Evaluación	11
4.5. Entorno Experimental	12
5. Resultados	12
5.1. Ontologías Artificiales	12
5.2. Fragmentos de <i>Galen</i>	13
6. Discusión y Análisis	14
7. Conclusiones	15
Repositorio y Reproducibilidad	15
Referencias	15
Appendices	18

A. Material Complementario	18
A.1. Resultados Completos: Ontologías Artificiales	18
A.2. Resultados Completos: Fragmentos de <i>Galen</i>	19

1. Introducción

1.1. Definición del Problema

1.1.1. Lógicas Descriptivas y Razonamiento

Las lógicas descriptivas son una familia de formalismos utilizados para la representación del conocimiento. En particular, *attributive language with complement* ($\mathcal{ALC}$) es un fragmento básico de estas, el cual cuenta con:

- **Conceptos atómicos:** que denotan conjuntos de individuos (ej. *Persona*).
- **Roles:** que denotan relaciones binarias entre individuos (ej. *Hijo*). En general, dado $R(x, y)$, x se denomina R -predecesor de y , mientras que este último se denomina R -sucesor de x , sucesor de x vía R , o bien, el *role-filler*.
- **Individuos:** nombres para elementos específicos del dominio $\mathcal{D}$ (ej. *Antonio*).

Adicionalmente, el lenguaje permite la construcción de conceptos más complejos a través de los constructores de la Tabla 1. Por ejemplo, sea la relación *Hijo*, el concepto *Apuesto*, los

Constructor	Sintaxis	Descripción
Top	$\top$	El concepto más general
Bottom	$\perp$	El concepto vacío
Negación	$\neg C$	Complemento del concepto C
Conjunción	$C \sqcap D$	Intersección de C y D
Disyunción	$C \sqcup D$	Unión de C y D
Restricción Existencial	$\exists R.C$	Individuos con al menos un sucesor vía R en C
Restricción de Valor	$\forall R.C$	Individuos cuyos sucesores vía R están todos en C

Tabla 1: Constructores de la lógica descriptiva $\mathcal{ALC}$.

individuos *Pedro*, *Guillermo* y *Pablo*. Supóngase que se tiene:

1. $Hijo(Pedro, Guillermo)$
2. $Hijo(Pedro, Pablo)$
3. $Apuesto(Guillermo)$
4. $\neg Apuesto(Guillermo)$

Se puede deducir:

1. $(\exists Hijo.Apuesto)(Pedro)$, ya que *Pedro* tiene como sucesor vía *Hijo* a *Guillermo*, y se sabe que $Apuesto(Guillermo)$. Es decir, “*Pedro* tiene un *Hijo Apuesto: Guillermo*”.

2. $(\neg \forall \text{Hijo.Apuesto})(\text{Pedro})$, ya que *Pedro* tiene como sucesor vía *Hijo* a *Pablo*, y se sabe que $\neg \text{Apuesto}(\text{Pablo})$. Es decir, “*Pedro* tiene un *Hijo* que no es *Apuesto: Pablo*”.

La inferencia básica sobre conceptos en lógicas descriptivas es la *subsunción*, escrita $C \sqsubseteq D$, que denota que D es más general que C , o bien, que los individuos de C forman parte de D bajo cualquier interpretación. Este trabajo se centra en la tarea de clasificación: dado un conjunto de conceptos y axiomas, esta busca encontrar todas las subsunciones entre los conceptos.

1.1.2. El Algoritmo de *Tableau*

Para mostrar $C \sqsubseteq D$, el algoritmo de *tableau* intenta mostrar que $C \sqcap \neg D$ es insatisfacible a través de la construcción incremental de un modelo hasta completarlo o llegar a una contradicción (*clash*) (Baader & Sattler, 2001; Schmidt-Schauß & Smolka, 1991). Este modelo toma la forma de un árbol (N, E) , donde cada nodo $n \in N$ representa un individuo con un conjunto de conceptos asociados $\mathcal{L}(n) = \{C_1, \dots, C_k\}$. Por su parte, cada arco $(n_1, n_2) \in E$ representa una relación de rol entre individuos. Esto es, si $(\exists R.C) \in \mathcal{L}(n_1)$, se infiere que $(n_1, n_2) \in E$, y además $C \in \mathcal{L}(n_2)$. Un nodo n puede tener tantos sucesores como restricciones existenciales haya en su etiqueta $\mathcal{L}(n)$, y estas ramas deben verificarse de forma conjunta: todas han de quedar libres de contradicción. En la Figura 1 se muestra un ejemplo que logra construir un modelo sin contradicciones.

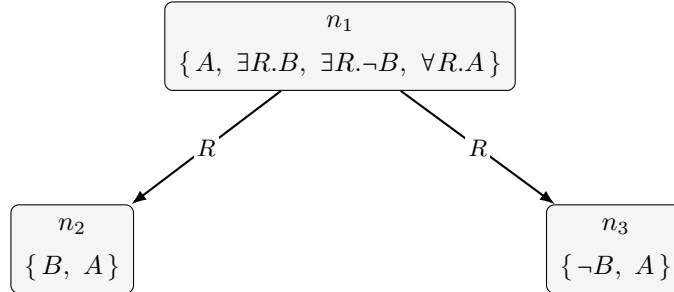


Figura 1: Árbol de completación para $A \sqcap \exists R.B \sqcap \exists R.\neg B \sqcap \forall R.A$. La raíz genera un sucesor por cada restricción existencial (n_2 y n_3), y la restricción universal $\forall R.A$ propaga A a ambos. Ninguna rama contiene un *clash*.

Visto de otra manera, el árbol de expansión es un árbol AND-OR: las disyunciones generan alternativas de las cuales basta que una sea libre de contradicción (ramas OR), mientras que los sucesores de rol deben verificarse todos (ramas AND). La Figura 2 ilustra esta estructura para el concepto $(\exists r_1.C_1 \sqcap \exists r_2.C_2) \sqcup (C_3 \sqcap \neg C_3)$, ejemplo tomado de Wu y Haarslev (2013a).

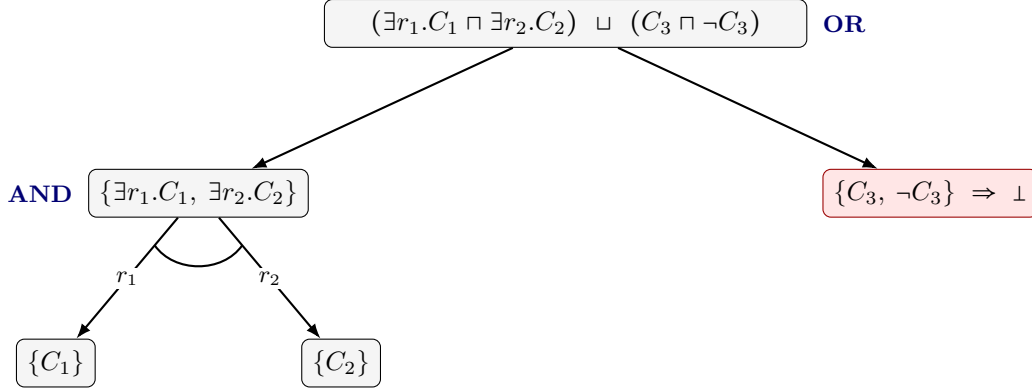


Figura 2: Estructura de árbol AND-OR subyacente al árbol de expansión.

En la práctica, la clasificación de una ontología con n conceptos atómicos induce del orden de n^2 tests de satisfacibilidad, cada uno de los cuales construye su propio árbol de expansión. La mayoría de estos tests resulta satisfacible, y en un test satisfacible *todas* las ramas conjuntivas de la rama disyuntiva exitosa deben explorarse de todas formas (Wu & Haarslev, 2013a). Es precisamente en ese trabajo obligatorio donde este proyecto busca introducir paralelismo.

1.2. Motivación e Importancia

El razonamiento en lógicas descriptivas (DL) es la forma en que se decide consistencia, satisfacibilidad y subsunción sobre ontologías descritas en un lenguaje como *OWL*. El método dominante para DLs expresivas es el algoritmo de *tableau* presentado anteriormente.

El problema es intrínsecamente costoso. Incluso considerando una DL relativamente sencilla como $\mathcal{ALC}$, la satisfacibilidad de conceptos es PSPACE-completa, y si se incluyen axiomas de inclusión general, pasa a ser EXPTIME-completa (Baader et al., 2003; Schmidt-Schauß & Smolka, 1991). Aún así, los razonadores logran buen desempeño empírico en problemas reales (Horrocks et al., 2000). Sin embargo, a medida que las ontologías crecen en tamaño y complejidad (web semántica, *big data*), los tiempos de clasificación pueden volverse prohibitivos.

La brecha que motiva el trabajo es esta: el *hardware* multinúcleo es omnipresente, pero los principales razonadores basados en *tableau* son de naturaleza secuencial (Glimm et al., 2014; Tsarkov & Horrocks, 2006; Wu & Haarslev, 2013a).

En base a lo anterior, el objetivo del proyecto es implementar y evaluar un razonador de *tableau* para $\mathcal{ALC}$ que paralelice el procesamiento de las ramas conjuntivas del árbol de expansión, tomando como referencia el trabajo de Wu y Haarslev (2013a). El resto del informe se organiza de la siguiente manera: la Sección 2 revisa el trabajo previo, la Sección 3 describe la solución propuesta y su implementación, la Sección 4 detalla la metodología experimental, la Sección 5 presenta los resultados, y las Secciones 6 y 7 los discuten y concluyen.

2. Trabajo Previo y Soluciones Existentes

2.1. Paralelización AND/OR

El árbol de expansión del *tableau* es un árbol AND-OR. En base a esto, ocurren dos tipos de ramificación cualitativamente distintos, que dan lugar a paralelismos diferentes:

- **Ramas disyuntivas (OR):** Proviene de la regla de la disyunción ($\sqcup$), no determinista. Desde un nodo sobre el que se aplica una regla $\sqcup$ se generan dos subárboles, basta que uno quede libre de contradicción (Baader & Sattler, 2001). Se trata de la principal fuente de complejidad del *tableau*, y es el objetivo natural del OR-*parallelism*.
- **Ramas conjuntivas (AND):** Proviene de los sucesores de rol generados por las restricciones existenciales ($\exists$). Son individuos independientes que deben verificarse todos sin *clash*. Son el objetivo del AND-*parallelism*.

En general, casi toda la investigación previa en *tableau* paralelo ataca las ramas disyuntivas, en particular:

- Liebig y Müller (2007): Trabajo seminal de paralelización de *tableau*. Su prototipo *UUPR* computa concurrentemente las elecciones no deterministas (ramas OR) en un modelo de memoria compartida. Reportan resultados preliminares prometedores frente a razonadores establecidos.
- Wu y Haarslev (2012): Presentan *Deslog*, un razonador *ALC* con OR-*parallelism*, también en memoria compartida (Java). Es el precursor directo del trabajo de los mismos autores sobre ramas conjuntivas (Wu & Haarslev, 2013a).

Por otra parte, el AND-*parallelism* se encuentra comparativamente inexplorado, un único trabajo lo aborda: Wu y Haarslev (2013a) paralelizan las ramas conjuntivas de profundidad cero en *ALC*. Utilizan un modelo de memoria compartida (Java, `java.concurrent`), con un *thread pool* de tamaño fijo y una cola concurrente que *bufferea* las ramas conjuntivas, implementado sobre *Deslog* (Wu & Haarslev, 2012). Sus experimentos sobre versiones escaladas artificialmente de una ontología sintética muestran que el beneficio esencialmente depende del número medio de ramas conjuntivas por test (τ , ver Sección 4): con $\tau \approx 2$ el *overhead* de las hebras anula cualquier ganancia, las mejoras aparecen con $\tau \geq 3$, y el mejor caso reportado alcanza un *speedup* de 12,6 con 15 hebras y $\tau \approx 29$ en una máquina de 16 núcleos. Los autores identifican como limitante principal el costo de las primitivas de concurrencia de bajo nivel de Java (colas concurrentes, variables *volatile*).

2.2. Paralelización de Granularidad más Gruesa

Otra familia de trabajos no paraleliza dentro del *tableau*, sino entre tareas de razonamiento independientes. Este es un eje ortogonal al de las ramas AND/OR:

- Aslani y Haarslev (2010): Paralelizan la tarea de clasificación distribuyendo los tests de subsunción entre hebras en memoria compartida. Se particiona el trabajo de construir la jerarquía de conceptos, no las ramas internas de cada *tableau*.
- Steigmiller, Glimm y Liebig (2014): Proponen *merge classification*, una estrategia de clasificación paralela por fusión de jerarquías parciales.
- *Konclude* Steigmiller, Liebig y Glimm, 2014: Razonador basado en *tableau*, paralelo y de alto rendimiento para *OWL 2 DL*. Su concurrencia opera principalmente a nivel de tests independientes (chequeos de consistencia y consultas), no de ramas conjuntivas dentro de un *tableau*. Es la referencia de razonador paralelo maduro.

2.3. Otros Paradigmas Concurrentes

Por último, es importante mencionar que *tableau* no es el único algoritmo utilizado para el razonamiento en DLs, ya que su desempeño es prohibitivo en ontologías de gran tamaño como *SNOMED-CT* (SNOMED International, 2026) o *Gene Ontology* (Ashburner et al., 2000), sobre las cuales se suelen utilizar métodos basados en consecuencias (*consequence-based*) o métodos híbridos. Por ejemplo:

- *ELK* Kazakov et al., 2014: Razonador *consequence-based* concurrente para el perfil $\mathcal{EL}$, extremadamente rápido en ontologías masivas como *SNOMED-CT*, a costa de una expresividad reducida.
- *Sequoia* Furmston et al., 2025: Extensión paralela reciente del razonador *consequence-based* *Sequoia* para DLs expresivas, que compara estrategias de paso de mensajes y *thread pools* en memoria compartida, reportando *speedups* de hasta 2,62.
- *WebPIE* Urbani et al., 2012: Motor de inferencia distribuido basado en *MapReduce* para fragmentos ligeros (RDFS, OWL-Horst) a escala web. Delimita el extremo opuesto del espectro: escala masiva a costa de expresividad.

2.4. Desafíos Identificados

De la revisión anterior se desprenden los principales desafíos que enfrenta la paralelización del *tableau*, y que condicionan el este trabajo:

- **Grafo de tareas dinámico e irregular:** el árbol de expansión se construye durante la ejecución y sus ramas tienen costos muy dispares, lo que dificulta una división estática del trabajo y motiva planificación dinámica.
- **Semánticas de terminación distintas:** en las ramas OR basta el primer éxito (terminación temprana al encontrar un modelo), mientras que en las ramas AND basta

el primer fracaso (cancelación al detectar un *clash*). Ambas requieren mecanismos de señalización entre hebras que introducen sincronización.

- **Duplicación de estado:** tanto el *backtracking* disyuntivo como la exploración paralela de ramas requieren copiar (o compartir de forma segura) el estado del árbol, un costo de memoria que puede anular el beneficio del paralelismo si se maneja ingenuamente.
- **Overhead de las primitivas de concurrencia:** Wu y Haarslev (2013a) identifican el costo de las estructuras concurrentes de bajo nivel (colas, *flags volatile*, gestión de memoria de la JVM) como su limitante principal, con $\tau \approx 2$ como umbral bajo el cual el paralelismo pierde frente al *overhead*.
- **Paralelismo acotado por la estructura del problema:** el AND-*parallelism* no puede escalar más allá del número de ramas conjuntivas disponibles ($s \leq \tau$), una propiedad intrínseca de la ontología y no del algoritmo.

3. Propuesta de Solución

3.1. El Algoritmo

En términos generales, se divide la tarea de satisfacibilidad en cuatro fases, dado el nodo raíz:

1. Se aplican todas las reglas AND posibles al nodo; si se ha llegado a un *clash* en este punto, se finaliza.
2. Si no se pueden aplicar más reglas AND, se aplican las reglas OR: para propósitos de este trabajo, se decide intentar primero con el operando izquierdo, y se procesa el nodo desde el paso 1.
3. Si no hay disyunciones pendientes, se generan todas las ramas conjuntivas posibles y se añaden a un vector *frontier*.
4. Finalmente, cada rama conjuntiva de *frontier* se procesa en paralelo.

El algoritmo propuesto es sustancialmente similar al de Wu y Haarslev (2013a), con la diferencia de que, como se verá más adelante, se permite además ajustar la granularidad del paralelismo.

A continuación se muestra el algoritmo en términos simples:

Algoritmo 1: PROCESSNODE: test de satisfacibilidad con ramas conjuntivas paralelas

Entrada: nodo n , arena A , profundidad d , ancestros $\mathcal{A}$, *flag* compartido *cancel*

Salida: **true** si el subárbol enraizado en n es libre de *clash*

```
1 if cancel  $\vee$  CLASH( $n, A$ ) then return false;
2 if BLOCKED( $n, \mathcal{A}, A$ ) then return true;
3 saturar reglas  $\sqcap$  y unfolding sobre  $\mathcal{L}(n)$  (fase AND);
4 if CLASH( $n, A$ ) then return false;
5 if existe  $(C \sqcup D) \in \mathcal{L}(n)$  pendiente then
6   | copiar estado en arena nueva  $A'$  (copy-on-write), agregar  $C$ ;
7   | if PROCESSNODE( $n, A', d, \mathcal{A}$ ) then return true;
8   | repetir con  $D$  (backtracking);
9 frontier  $\leftarrow$  aplicar reglas  $\exists$  y propagar reglas  $\forall$  (sucesores de rol);
10 if frontier =  $\emptyset$  then return true;
11 if  $d < \text{MAX\_OMP\_DEPTH} \wedge |frontier| \geq 2$  then
12   | foreach rama  $c \in frontier$  en paralelo do
13   |   | ligar arena local al prefijo de  $A$ ;
14   |   | if  $\neg \text{PROCESSNODE}(c, A_{local}, d + 1, \mathcal{A} \cup \{n\})$  then cancel  $\leftarrow$  true;
15 else
16   | procesar cada  $c \in frontier$  secuencialmente, abortando al primer clash;
17 return  $\neg cancel$ ;
```

3.2. Implementación en *OpenMP*

En términos de implementación, el paralelismo se realiza a través de *OpenMP* en C++. En particular, se utiliza la directiva `#pragma omp parallel for`:

```
1 if (depth < MAX_OMP_DEPTH && frontier.size() >= 2) {
2   #pragma omp parallel for schedule(dynamic, CHUNK_SIZE) shared(global_clash)
3   for (size_t i = 0; i < frontier.size(); ++i) {
4     if (global_clash.load(std::memory_order_relaxed))
5       continue;
6
7     ScopedArena task_scoped(pool);
8     ThreadLocalArena& task_arena = task_scoped.get();
9     task_arena.bindPrefix(arena, arena.saveState(), frontier[i]);
10
11     std::vector<size_t> local_ancestors = ancestors;
12     if (!processNode(frontier[i], task_arena, depth + 1,
13                     local_ancestors, defs, &global_clash)) {
14       global_clash.store(true, std::memory_order_relaxed);
15     }
16   }
```

```

17 } else {
18     for (size_t child_idx : frontier) {
19         if (global_clash.load(std::memory_order_relaxed))
20             break;
21         if (!processNode(child_idx, arena, depth + 1, ancestors, defs,
22                         cancel)) {
23             global_clash.store(true, std::memory_order_relaxed);
24         }
25     }
26 }

```

Listing 1: Fase 4, cada rama conjuntiva de la frontera se procesa en paralelo.

El uso de esta directiva está justificado en la asimetría de las resoluciones de ramas conjuntivas. Además, nótese que se introducen los parámetros `MAX_OMP_DEPTH` y `CHUNK_SIZE`, que permiten definir la profundidad mínima del árbol a la que comenzar a paralelizar, y la cantidad de ramas conjuntivas asignadas a cada hebra, respectivamente.

3.3. Manejo de Memoria

La aplicación de reglas `OR` y `∃` implica la creación de nuevos nodos en el árbol, para los cuales se debe duplicar el estado de este. De no manejarse con cuidado, esto podría significar un *overhead* que anularía cualquier beneficio otorgado por el paralelismo. La manera en que se resuelve esto es a través de *thread-local arenas*: cada hebra posee una arena de memoria en la cual puede crear nodos y asignar conceptos de forma contigua, el *overhead* de esta primera operación es minimizado a través del uso de prefijos: en lugar de duplicar el estado completo del árbol, la arena nueva registra un puntero a la arena madre junto con el estado de sus contadores al momento de la copia (método `bindPrefix`). Todo lo asignado antes de ese punto se considera un prefijo inmutable, y solo el nodo sobre el que se ramifica se copia efectivamente a la arena nueva. Se trata, en esencia, de un esquema *copy-on-write* de costo $O(|\mathcal{L}(n)|)$ por ramificación, independiente del tamaño del árbol acumulado.

La clase `ParallelReasoner` cuenta con un *pool* de arenas reutilizables, las cuales asigna según corresponda:

```

1 // El pool member se default-construye (lazy, sin arenas pre-alocadas)
2 explicit ParallelReasoner() = default;
3
4 bool isSatisfiable(const Concept* query,
5                   const std::unordered_map<const Concept*, const Concept
6                   *>& defs,
7                   const Concept* tbox = nullptr) override {
8     tbox_ = tbox;
9     ScopedArena master_scoped(pool);
10    ThreadLocalArena& arena = master_scoped.get();

```

```

11     size_t root_idx = arena.allocateNode();
12     arena.addLabelToNode(root_idx, query);
13     if (tbox_)
14         arena.addLabelToNode(root_idx, tbox_);
15
16     // El nodo raiz no tiene ancestros, logicamente
17     // processNode manejara pushing y popping a medida que el arbol crece
18     std::vector<size_t> root_ancestors;
19
20     // Invocamos el dispatcher maestro
21     return processNode(root_idx, arena, 0, root_ancestors, defs);
22 }
23 };

```

Listing 2: ParallelReasoner adquiere arenas desde un pool reutilizable (construccion *lazy*).

4. Metodología

4.1. Descripción de los Datos

Para la evaluación de la solución propuesta se contemplan dos tipos de ontologías: artificiales, que buscan inflar el número de ramas conjuntivas, con el fin de estudiar las condiciones mínimas necesarias para obtener beneficios del AND-*parallelism*, y reales, con el propósito de evaluar la viabilidad de la solución en tareas de razonamiento reales.

4.1.1. Tea Ontology Artificial

Wu y Haarslev (2013a) proponen una métrica que predice la eficiencia del AND-*parallelism*: dado un test de satisfacibilidad, se define μ como el número de ramas conjuntivas involucradas, y N_μ el número total de tests que procesan exactamente μ ramas conjuntivas para un experimento específico. Luego, se define τ como:

$$\tau = \frac{\sum_{\mu=0}^k N_\mu \times \mu}{\sum_{\mu=0}^k N_\mu}$$

para algún $k \in \mathbb{N}$. Es decir, se interpreta como el número de ramas conjuntivas promedio por test de satisfacibilidad.

En base a esto, Wu y Haarslev (2013a) crean un *benchmark* de ontologías artificiales construidas desde *tea ontology* (Wu & Haarslev, 2013b): para j dado, definimos *tea_j* como *tea ontology* con el axioma de inclusión general:

$$\bigwedge_{i=0}^j \forall R_{2i+1}.(C_{2i+1} \sqcap C_{2i+2}) \sqsubseteq \bigwedge_{i=0}^j \exists R_{2i}.(C_{2i} \sqcup C_{2i+1})$$

Al agregar este axioma, se introducen ramas conjuntivas no triviales en los tests de satisfaci-
bilidad de *tea_j*.

4.1.2. Ontología *Galen*

La ontología *Galen* es una ontología médica de gran escala desarrollada en el proyecto europeo homónimo para la representación de terminología clínica, y es un *benchmark* clásico para razonadores DL por su tamaño y estructura fuertemente cíclica (Rector & Rogers, 2006). La versión utilizada (*full-Galen*) contiene 23 141 conceptos y 950 roles, y su expresividad excede $\mathcal{ALC}$: incluye, entre otros, jerarquías de roles, roles transitivos e inversos.

La tarea de clasificación sobre *Galen* resulta intratable para un razonador de *tableau* no optimizado, sin mencionar que esta es más expresiva que $\mathcal{ALC}$. Es por esto que se construyeron subconjuntos de *Galen* reducidos a $\mathcal{ALC}$ formados alrededor del concepto *Heart*: definimos *Galen_n* como la ontología de n conceptos más cercanos a *Heart* (incluyéndolo), computados a través de una búsqueda por anchura. Vale notar que debido a la monotonía de la implicación, cualquier inferencia hecha sobre uno de estos fragmentos de *Galen* es válida para *Galen* en sí (reducida a $\mathcal{ALC}$).

4.1.3. Datos Finales

En concreto, la experimentación incluyó tests de clasificación para *tea_j* con $j \in \{2, 3, \dots, 10\}$, además de *galen_50*, *galen_75* y *galen_100*. En la Tabla 2 se muestra un resumen de escala de las ontologías artificiales, mientras que los fragmentos de *Galen* se resumen en la Tabla 3. Los valores de τ fueron medidos a través del razonador secuencial, que registra μ para cada test de subsunción.

j	Conceptos atómicos	Axiomas (internalizados)	τ
2	22	34	2,10
3	23	35	3,04
4	24	36	3,04
5	25	37	4,00
6	26	38	4,00
7	27	39	4,95
8	28	40	4,96
9	29	41	5,92
10	30	42	5,92

Tabla 2: Resumen de las ontologías artificiales *tea_j*. Nótese que τ crece cada dos valores de j : solo las restricciones existenciales (índices pares del axioma) generan sucesores de rol.

Fragmento	Conceptos	Axiomas (internalizados)	τ
<i>galen_50</i>	50	118	2,13
<i>galen_75</i>	75	160	1,96
<i>galen_100</i>	100	197	1,76

Tabla 3: Resumen de los fragmentos $\mathcal{ALC}$ de *Galen* centrados en el concepto *Heart*.

4.2. Baseline

La *baseline* de todos los experimentos es el razonador secuencial (**SerialReasoner**), que implementa exactamente el mismo cálculo de *tableau*, las mismas optimizaciones (absorción de axiomas, *lazy unfolding*, *blocking* por ancestros) y el mismo esquema de arenas, pero sin directivas *OpenMP*. El tiempo medio de clasificación de esta versión con una hebra define T_1 .

Adicionalmente, se midió la versión paralela ejecutada con una sola hebra (**ParallelReasoner**, $p = 1$), con el propósito de aislar el *overhead* estructural introducido por el camino de código paralelo (adquisición de arenas, *flags* de cancelación) independientemente de la concurrencia efectiva. Cada configuración se ejecutó 5 veces para las ontologías artificiales, y 3 veces para las reales. Se reporta la media y la desviación estándar.

4.3. Hiperparámetros

Todos los experimentos fijan los dos hiperparámetros del razonador paralelo introducidos en la Sección 3: `MAX_OMP_DEPTH = 1`, solo la frontera de profundidad cero se procesa en paralelo (el mismo alcance que el trabajo de referencia), y `CHUNK_SIZE = 1` con planificación `schedule(dynamic)`, asignando las ramas conjuntivas de a una por hebra. El número de hebras $p \in \{1, 2, 3, 4\}$ se controla mediante `omp_set_num_threads`. No se realizó una búsqueda exhaustiva sobre estos hiperparámetros, su exploración queda como trabajo futuro (Sección 7).

4.4. Métricas de Evaluación

Para cada tarea de clasificación se calcularon los tiempos de clasificación en milisegundos, y en base a estos se computó el *speedup* en comparación al tiempo de la *baseline*, T_1 :

$$S_p = \frac{T_1}{T_p}$$

y la eficiencia:

$$E_p = \frac{S_p}{p}$$

4.5. Entorno Experimental

Todos los experimentos fueron ejecutados en un sistema con las especificaciones de la Tabla 4.

Componente	Especificación
CPU	Intel Core i5-7300HQ 2,50 GHz (4 núcleos, 4 hebras, 6144 KB de cache)
Caché	L1: 64 KB, L2: 256 KB, L3: 6 MB (compartido entre núcleos)
Memoria RAM	16 GB
Sistema operativo	Fedora release 42 (Adams)
Compilador	g++ versión 15.2.1 (Red Hat 15.2.1-7)

Tabla 4: Entorno de ejecución utilizado en la evaluación.

5. Resultados

5.1. Ontologías Artificiales

La Tabla 5 resume los resultados de clasificación sobre las ontologías *tea_j*: tiempo secuencial medio y *speedup* de la versión paralela con $p \in \{1, 2, 3, 4\}$ hebras, junto a los valores reportados por Wu y Haarslev (2013a) a τ equivalente. Las tablas completas, incluyendo desviaciones estándar y eficiencias, se encuentran en el Anexo A.

j	τ	T_1 (ms)	Este trabajo (S_p)				Wu y Haarslev, 2013a (S_p) ¹	
			$p = 1$	$p = 2$	$p = 3$	$p = 4$	$p = 2$	$p = 3$
2	2,10	31,39	0,93	1,59	1,26	0,60	0,75	0,76
3	3,04	149,45	0,99	1,59	1,93	1,46	1,05	1,18
4	3,04	160,17	1,00	1,61	1,88	1,50	1,05	1,18
5	4,00	1 016,44	1,03	1,75	2,04	2,34	–	–
6	4,00	1 087,77	1,03	1,75	2,05	2,27	–	–
7	4,95	8 261,91	1,03	1,75	2,32	2,27	1,30	1,52
8	4,96	8 835,69	1,03	1,74	2,32	2,27	1,30	1,52
9	5,92	76 287,76	1,03	1,84	2,53	2,53	–	–
10	5,92	80 813,00	1,03	1,83	2,52	2,54	–	–

Tabla 5: Tiempo secuencial y *speedup* de clasificación sobre *tea_j* (media de 5 ejecuciones), junto a los valores del trabajo de referencia a τ equivalente (2,09; 3,09; 5,08). En negrita, el mejor valor de cada comparación.

¹Valores emparejados por τ en referencia a Wu y Haarslev (2013a). El trabajo original no publica tablas numéricas, sus valores fueron extraídos de las figuras del artículo.

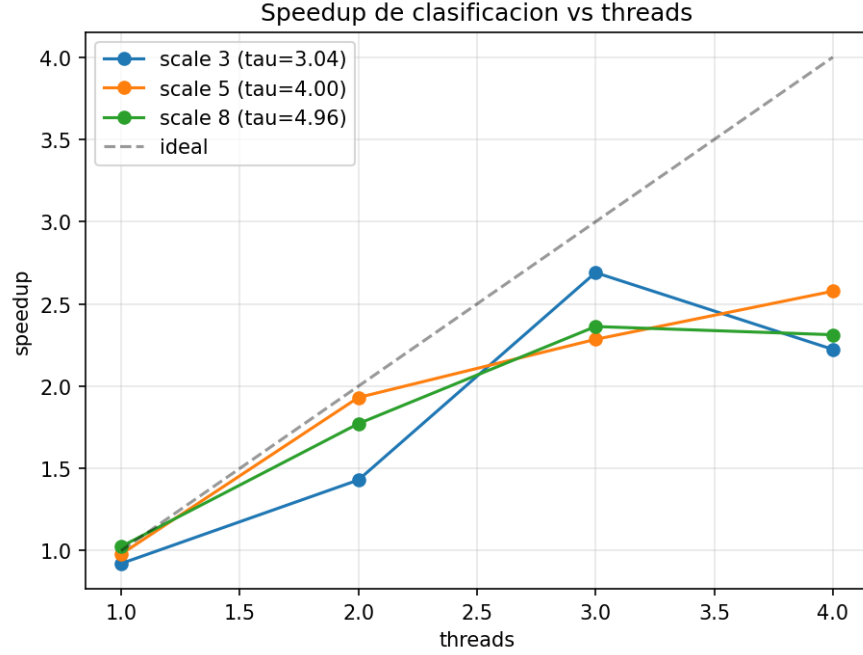


Figura 3: Speedup en función del número de hebras para las ontologías artificiales *tea_j*.

5.2. Fragmentos de *Galen*

La Tabla 6 presenta los resultados análogos para los fragmentos reales de *Galen*.

Fragmento	τ	T_1 (ms)	Speedup S_p			
			$p = 1$	$p = 2$	$p = 3$	$p = 4$
<i>galen_50</i>	2,13	1 300,59	0,95	1,27	1,25	1,21
<i>galen_75</i>	1,96	34 164,67	0,99	1,00	0,89	0,85
<i>galen_100</i>	1,76	1 342 003,33	1,00	1,22	1,11	1,11

Tabla 6: Tiempo secuencial y *speedup* de clasificación sobre los fragmentos de *Galen* (media de 3 ejecuciones).

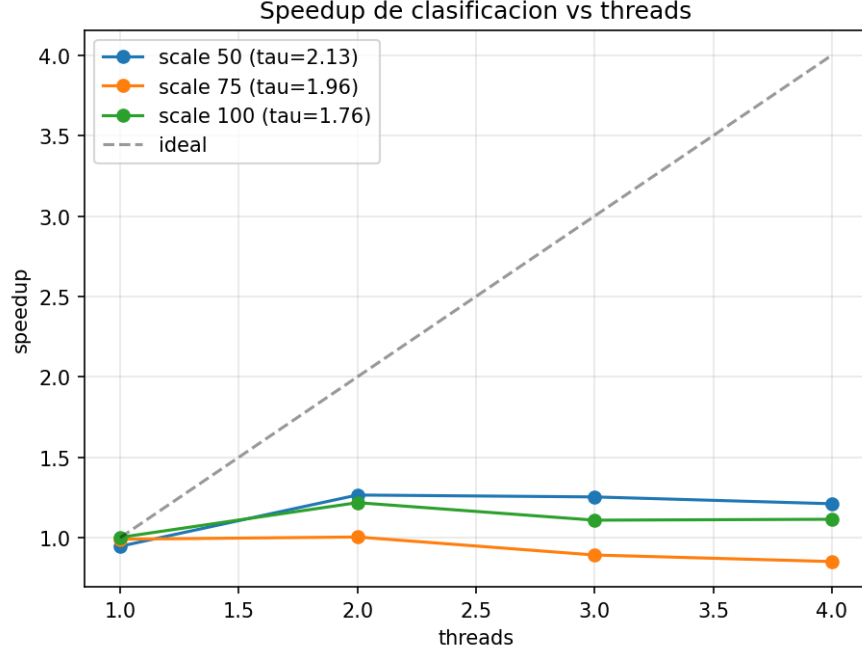


Figura 4: *Speedup* en función del número de hebras para los fragmentos de *Galen*.

6. Discusión y Análisis

La discusión se desarrolla en torno a tres ejes: la cota estructural que impone τ sobre la escalabilidad, la comparación con el trabajo de referencia a igual τ , y el comportamiento sobre ontologías reales. En primer lugar, los datos confirman la cota propuesta por Wu y Haarslev (2013a), esto es, $s \leq \tau$ en todos los casos. En general, se puede extraer una variedad de conclusiones en base a τ : $p = 4$ no mejora el *speedup* para $\tau < 5$, y obtiene mejoras marginales en $\tau \approx 5,9$. Esto se puede atribuir en parte al hecho de que el entorno experimental contaba con cuatro núcleos físicos, y la ley de Amdahl: si las ramas conjuntivas son escasas, la sección secuencial domina los tiempos y la paralelización provee escasos beneficios. Por otro lado, Wu y Haarslev (2013a) establecen que el *overhead* anula las ganancias para $\tau \approx 2,1$, con un *speedup* de $\sim 0,75$, sin embargo, en este trabajo se obtiene un *speedup* de 1,59 con 2 hebras. Esto se podría atribuir a la arquitectura de memoria y primitivas de concurrencia: las arenas *thread-local* junto al uso de prefijos y *OpenMP* resultarían superiores frente a `java.concurrent`. Es decir, el punto en el que el AND-*parallelism* se vuelve rentable en base a τ depende en gran parte de la implementación. En general, el *speedup* es sublineal, con una eficiencia $E_2 = 0,80\text{--}0,92$, lo que contrasta con el mejor caso de 0,84 alcanzado por Wu y Haarslev (2013a), recién con 15 hebras a un $\tau \approx 29$. Finalmente, existen algunas anomalías, por ejemplo, en *tea_2* con $p = 4$ se consiguen desviaciones estándar grandes, de 32,57 ms con una media de ~ 52 ms. Esto se podría atribuir a que el trabajo por test es minúsculo: $\tau \approx 2$ sugiere hebras compitiendo por trabajo inexistente.

Con respecto a los fragmentos de *Galen*, τ se mantiene menor a 2,2 en los tres casos. Esto explicaría las pocas ganancias ($S_2 \leq 1,27$). Se puede deducir que las ontologías reales como *Galen* ofrecen poco AND-*parallelism* de profundidad cero. De igual manera, existe la posibilidad de que esto sea un sesgo de los fragmentos generados por una búsqueda por anchura desde *Heart*.

Finalmente, el costo estructural del paralelismo es mínimo, $S_1 \approx 0,93-1,03$ en general. Es decir, el costo fijo de las arenas y *flags* es despreciable.

7. Conclusiones

Se implementó un razonador de *tableau* paralelo para *ALC* utilizando C++/*OpenMP* enfocado en el procesamiento concurrente de ramas conjuntivas, y se evaluó sobre el *benchmark* del trabajo de Wu y Haarslev (2013a) y fragmentos de *Galen*. Se obtuvo un *speedup* máximo de 2,54 con 4 hebras y 2,53 con 3, además de mejoras con respecto a Wu y Haarslev (2013a) a igual τ en todos los puntos medidos. Se confirma empíricamente $s \leq \tau$ como límite del AND-*parallelism*. En fragmentos reales de *Galen*, el τ bajo limita los beneficios de este tipo de paralelismo, justificando el enfoque de la literatura al utilizar razonadores híbridos y paralelización externa a los tests de satisfacibilidad.

Como trabajo futuro, se propone realizar OR-*parallelism* sobre la misma arquitectura de memoria, paralelizar el *loop* externo de clasificación, combinarlo con paralelismo AND/OR, y realizar una búsqueda exhaustiva de valores de MAX_OMP_DEPTH y CHUNK_SIZE óptimos. Finalmente, se propone evaluar τ sobre un corpus amplio de ontologías reales para caracterizar la rentabilidad del AND-*parallelism* en estas.

Repositorio y Reproducibilidad

La implementación se encuentra disponible en: <https://github.com/Askorin/alc-tableau-parallel>. El archivo README.md cuenta con instrucciones de compilación y ejecución de los *benchmarks*.

Referencias

- Ashburner, M., Ball, C. A., Blake, J. A., Botstein, D., Butler, H., Cherry, J. M., Davis, A. P., Dolinski, K., Dwight, S. S., Eppig, J. T., et al. (2000). Gene Ontology: tool for the unification of biology. *Nature Genetics*, 25(1), 25-29. <https://doi.org/10.1038/75556>
- Aslani, M., & Haarslev, V. (2010). Parallel TBox Classification in Description Logics – First Experimental Results. *Proceedings of the 19th European Conference on Artificial Intelligence (ECAI 2010)*, 485-490.

- Baader, F., Calvanese, D., McGuinness, D. L., Nardi, D., & Patel-Schneider, P. F. (Eds.). (2003). *The Description Logic Handbook: Theory, Implementation and Applications*. Cambridge University Press.
- Baader, F., & Sattler, U. (2001). An Overview of Tableau Algorithms for Description Logics. *Studia Logica*, 69(1), 5-40. <https://doi.org/10.1023/A:1013882326814>
- Furmston, A., Tena Cucala, D. J., Chen, J., & Cuenca Grau, B. (2025). Parallel Reasoning in Sequoia. *The Semantic Web – ISWC 2025*. https://doi.org/10.1007/978-3-032-09527-5_17
- Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). HermiT: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 53(3), 245-269. <https://doi.org/10.1007/s10817-014-9305-1>
- Horrocks, I., Sattler, U., & Tobies, S. (2000). Practical Reasoning for Very Expressive Description Logics. *Logic Journal of the IGPL*, 8(3), 239-263. <https://doi.org/10.1093/jigpal/8.3.239>
- Kazakov, Y., Krötzsch, M., & Simančík, F. (2014). The Incredible ELK: From Polynomial Procedures to Efficient Reasoning with EL Ontologies. *Journal of Automated Reasoning*, 53(1), 1-61. <https://doi.org/10.1007/s10817-013-9296-3>
- Liebig, T., & Müller, F. (2007). Parallelizing Tableaux-Based Description Logic Reasoning. *On the Move to Meaningful Internet Systems 2007: OTM 2007 Workshops, Part II*, 1135-1144. https://doi.org/10.1007/978-3-540-76890-6_41
- Rector, A. L., & Rogers, J. (2006). Ontological and Practical Issues in Using a Description Logic to Represent Medical Concept Systems: Experience from GALEN. *Reasoning Web, Second International Summer School*, 197-231. https://doi.org/10.1007/11837787_9
- Schmidt-Schauß, M., & Smolka, G. (1991). Attributive concept descriptions with complements. *Artificial Intelligence*, 48(1), 1-26. [https://doi.org/10.1016/0004-3702\(91\)90078-X](https://doi.org/10.1016/0004-3702(91)90078-X)
- SNOMED International. (2026). *SNOMED CT*. Consultado el 15 de julio de 2026, desde <https://www.snomed.org>
- Steigmiller, A., Glimm, B., & Liebig, T. (2014). Parallel OWL Reasoning: Merge Classification. *Semantic Technology – Joint International Semantic Technology Conference (JIST)*.
- Steigmiller, A., Liebig, T., & Glimm, B. (2014). Konclude: System Description. *Journal of Web Semantics*, 27–28, 78-85. <https://doi.org/10.1016/j.websem.2014.06.003>
- Tsarkov, D., & Horrocks, I. (2006). FaCT++ Description Logic Reasoner: System Description. *Automated Reasoning – Third International Joint Conference (IJCAR 2006)*, 292-297. https://doi.org/10.1007/11814771_26
- Urbani, J., Kotoulas, S., Maassen, J., van Harmelen, F., & Bal, H. (2012). WebPIE: A Web-scale Parallel Inference Engine using MapReduce. *Journal of Web Semantics*, 10, 59-75. <https://doi.org/10.1016/j.websem.2011.05.004>

- Wu, K., & Haarslev, V. (2012). A Parallel Reasoner for the Description Logic ALC. *Proceedings of the 2012 International Workshop on Description Logics (DL 2012)*, 846.
- Wu, K., & Haarslev, V. (2013a). Exploring Parallelization of Conjunctive Branches in Tableau-based Description Logic Reasoning. *Proceedings of the 2013 International Workshop on Description Logics (DL 2013)*, 1014.
- Wu, K., & Haarslev, V. (2013b). *Tea ontology (material suplementario de Deslog)*. Consultado el 15 de julio de 2026, desde <http://code.google.com/p/deslog/downloads/detail?name=tea.tar.gz>

Anexos

A. Material Complementario

A.1. Resultados Completos: Ontologías Artificiales

j	Modo (p)	Media (ms)	Desv. est. (ms)	Speedup	Eficiencia
2	serial (1)	31,39	0,46	1,000	–
2	parallel (1)	33,82	0,53	0,928	0,928
2	parallel (2)	19,75	0,57	1,589	0,795
2	parallel (3)	24,95	4,33	1,258	0,419
2	parallel (4)	52,17	32,57	0,602	0,150
3	serial (1)	149,45	1,65	1,000	–
3	parallel (1)	150,60	0,76	0,992	0,992
3	parallel (2)	93,81	3,05	1,593	0,797
3	parallel (3)	77,64	6,20	1,925	0,642
3	parallel (4)	102,61	3,86	1,456	0,364
4	serial (1)	160,17	1,83	1,000	–
4	parallel (1)	159,49	1,79	1,004	1,004
4	parallel (2)	99,36	1,01	1,612	0,806
4	parallel (3)	85,16	4,25	1,881	0,627
4	parallel (4)	106,71	1,16	1,501	0,375
5	serial (1)	1 016,44	4,52	1,000	–
5	parallel (1)	988,59	3,21	1,028	1,028
5	parallel (2)	579,92	2,28	1,753	0,876
5	parallel (3)	497,51	2,16	2,043	0,681
5	parallel (4)	434,60	5,72	2,339	0,585
6	serial (1)	1 087,77	4,73	1,000	–
6	parallel (1)	1 057,04	3,34	1,029	1,029
6	parallel (2)	620,25	1,05	1,754	0,877
6	parallel (3)	531,25	4,51	2,048	0,683
6	parallel (4)	479,50	23,02	2,269	0,567
7	serial (1)	8 261,91	23,31	1,000	–
7	parallel (1)	8 002,13	9,76	1,032	1,032
7	parallel (2)	4 729,55	9,27	1,747	0,873
7	parallel (3)	3 558,72	13,49	2,322	0,774
7	parallel (4)	3 633,64	38,37	2,274	0,568
8	serial (1)	8 835,69	14,39	1,000	–
8	parallel (1)	8 561,10	22,68	1,032	1,032
8	parallel (2)	5 067,21	15,44	1,744	0,872
8	parallel (3)	3 812,45	21,12	2,318	0,773
8	parallel (4)	3 896,87	15,24	2,267	0,567
9	serial (1)	76 287,76	114,71	1,000	–
9	parallel (1)	73 984,22	143,42	1,031	1,031
9	parallel (2)	41 535,86	110,11	1,837	0,918
9	parallel (3)	30 215,56	86,87	2,525	0,842
9	parallel (4)	30 196,38	299,67	2,526	0,632
10	serial (1)	80 813,00	56,02	1,000	–
10	parallel (1)	78 608,38	174,47	1,028	1,028
10	parallel (2)	44 098,62	106,97	1,833	0,916
10	parallel (3)	32 070,08	94,33	2,520	0,840
10	parallel (4)	31 869,12	163,18	2,536	0,634

Tabla 7: Resultados completos sobre *tea_j*: media y desviación estándar de 5 ejecuciones.

A.2. Resultados Completos: Fragmentos de *Galen*

Fragmento	Modo (p)	Media (ms)	Desv. est. (ms)	Speedup	Eficiencia
<i>galen_50</i>	serial (1)	1 300,59	86,56	1,000	–
<i>galen_50</i>	parallel (1)	1 374,12	21,61	0,946	0,946
<i>galen_50</i>	parallel (2)	1 027,84	23,15	1,265	0,633
<i>galen_50</i>	parallel (3)	1 037,52	47,35	1,254	0,418
<i>galen_50</i>	parallel (4)	1 073,92	100,92	1,211	0,303
<i>galen_75</i>	serial (1)	34 164,67	38,66	1,000	–
<i>galen_75</i>	parallel (1)	34 490,23	171,07	0,991	0,991
<i>galen_75</i>	parallel (2)	34 018,80	105,38	1,004	0,502
<i>galen_75</i>	parallel (3)	38 287,00	469,70	0,892	0,297
<i>galen_75</i>	parallel (4)	40 117,17	615,96	0,852	0,213
<i>galen_100</i>	serial (1)	1 342 003,33	5 462,99	1,000	–
<i>galen_100</i>	parallel (1)	1 339 366,67	1 521,23	1,002	1,002
<i>galen_100</i>	parallel (2)	1 101 953,33	35 476,00	1,218	0,609
<i>galen_100</i>	parallel (3)	1 210 156,67	35 401,14	1,109	0,370
<i>galen_100</i>	parallel (4)	1 204 136,67	28 051,05	1,114	0,279

Tabla 8: Resultados completos sobre los fragmentos de *Galen*: media y desviación estándar de 3 ejecuciones.